QBIT9 Backend and Production Plan

Oct 22, 2026 · @Suman Yadav

1. Where the app stands

The frontend is finished against a mock backend; the backend job is to replace one folder (`src/services/mock`) with a real adapter, without touching a single screen.

What exists today (Next.js 16, React 19, TypeScript strict, Tailwind v4, TanStack Query, zod):

- Customer app: 20 screens (join queue with location proof, check-in, called, serving, feedback, booking, history, profile).
- Staff console under `/staff` : sign in, start shift, counter, queue, walk-in, today.
- One queue: while a staff counter is open, the customer's place is the real list staff work (`QueueContext.live`). A backend must always behave this way.
- 207 unit tests and 12 Playwright tests pass against the mock.

The seam. Screens call hooks, hooks call `Services` (`src/services/types.ts` for customers, `src/services/staff-types.ts` for staff), and `services/registry.ts` picks the adapter from `NEXT_PUBLIC_QBIT9_ADAPTER` . The `supabase` branch currently throws. That throw is what you replace.

What the mock fakes, and the backend must do for real:

Mock does	Backend must do
localStorage as database	Postgres tables with row level security
OTP code shown on screen (demo chip)	Real OTP over email, and SMS later
Location proof trusted from the browser	Server checks distance, accuracy and age of the proof
Timers move the line when no staff are in	Only staff actions call people; server timers only for windows
Polling and same-tab events	Realtime subscriptions and Web Push
Sample staff accounts, PIN 4821	Hashed PINs, rosters, lockout, audit log
Simulated crowd on the floor	Real tokens only
Policy numbers in `domain/policy.ts`	Per-branch config confirmed by the bank

2. Target architecture and stack

Use Supabase (Postgres, Auth, Realtime, Edge Functions) for the backend and keep the Next.js app on Vercel. It matches the existing seam (`services/supabase/` is already reserved), needs no server of your own to run, and gives Realtime and row level security without extra services. Nothing in the app forces this choice; any backend that implements the two service interfaces works.

How a request flows

1. A screen calls a hook (`useJoin`, `useCallNext`, …).
2. The hook calls `Services` from context, which is the Supabase adapter once `NEXT_PUBLIC_QBIT9_ADAPTER=supabase`.
3. The adapter calls a Postgres RPC (state changes) or a view (reads) through supabase-js. State changes never go through plain table writes.
4. The RPC runs the rules in one transaction, writes the row and an audit entry, and returns the fresh view.
5. Realtime pushes the change to every open screen (customer tab and staff counters). Web Push reaches customers whose app is closed.
6. A scheduled function closes expired call windows and no-shows.

Recommended stack

Concern	Choice	Why
Database, RLS, RPCs	Supabase Postgres (Mumbai region)	Transactions for queue order, RLS for isolation, data stays in India
Customer sign in	Supabase Auth, email OTP now, phone OTP later	Already shaped like the mock `AuthService`
Staff sign in	Edge Function: employee ID + PIN (argon2id), returns a short-lived JWT with staff claims	PINs are 4 to 6 digits, so lockout and hashing must be ours, not a password form
Live updates	Supabase Realtime (postgres changes)	Replaces polling for both apps
Background alerts	Web Push (VAPID) from an Edge Function; `public/sw.js` already handles push	Alerts with the tab closed
Email	Resend, or Supabase's own SMTP for the first pilot	OTP and booking emails

Concern	Choice	Why
SMS (later)	MSG91 or Twilio with India DLT registration	Gated by `NEXT_PUBLIC_FEATURE_SMS`
Timers	`pg_cron` calling an Edge Function every minute	Close lapsed calls, expire check-ins, mark no-shows
Hosting	Vercel for the Next.js app	`NEXT_PUBLIC_*` flags are baked at build
Errors and traces	Sentry (frontend and functions)	See section 9

3. Accounts, APIs and keys

You need six accounts for a pilot (Supabase, Vercel, Resend, Sentry, a domain registrar, GitHub) and two more before launch to real customers (an SMS provider, and the bank's data). Everything marked secret stays server side: Edge Function secrets, Vercel server env, or CI secrets. It never gets a `NEXT_PUBLIC_` prefix.

Service	Used for	Keys and values you need	Public or secret
Supabase	Database, Auth, Realtime, Edge Functions, cron	Project URL; publishable (anon) key; service role key; JWT secret; database connection string	URL and publishable key public. The other three secret
Vercel	Hosting the Next.js app	Project, domain, env vars per environment	Env values secret except `NEXT_PUBLIC_*`
Resend (or SMTP)	OTP and booking emails	API key; verified sending domain (SPF, DKIM, DMARC DNS records)	Secret
Web Push (VAPID)	Alerts with the app closed	One keypair: public key, private key, `mailto:` subject	Public key public, private secret

Service	Used for	Keys and values you need	Public or secret
SMS provider (MSG91 or Twilio)	Phone OTP and SMS alerts, later	Auth key; sender ID; India DLT entity ID and approved template IDs	Secret
Sentry	Error tracking	DSN; auth token for source maps	DSN public, token secret (CI)
Uptime monitor (Better Stack or UptimeRobot)	Alert when the site or a function is down	Monitor URLs, alert contacts	None
Domain and DNS	`app.yourdomain` and email records	Registrar login	None
GitHub	Source, CI, migrations	Repo; Actions secrets	Secret

The app needs **no maps API key**. Distance checks use the browser's geolocation and the branch coordinates stored in the database.

Environment variables to add (read them only in `src/lib/env.ts`, by their full name, because Next inlines `NEXT_PUBLIC_*` at build time):

```
# Vercel, public
NEXT_PUBLIC_QBIT9_ADAPTER=supabase
NEXT_PUBLIC_APP_URL=https://app.example.com
NEXT_PUBLIC_SUPABASE_URL=https://<project>.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=<publishable key>
NEXT_PUBLIC_VAPID_PUBLIC_KEY=<vapid public>
NEXT_PUBLIC_SENTRY_DSN=<dsn>
NEXT_PUBLIC_ENABLE_DEVTOOLS=false
NEXT_PUBLIC_ENABLE_STAFF=true
NEXT_PUBLIC_FEATURE_SMS=false

# Supabase Edge Function secrets (never in the browser)
SUPABASE_SERVICE_ROLE_KEY=<service role>
STAFF_JWT_SECRET=<project JWT secret>
VAPID_PRIVATE_KEY=<vapid private>
VAPID_SUBJECT=mailto:ops@example.com
RESEND_API_KEY=<key>
MSG91_AUTH_KEY=<key>        # later
```

```
# CI only
SUPABASE_ACCESS_TOKEN=<token>
SUPABASE_DB_PASSWORD=<password>
SENTRY_AUTH_TOKEN=<token>
```

Generate the VAPID keypair once with `npx web-push generate-vapid-keys` and never rotate it casually: every stored browser subscription is tied to it.

Data you must get from the bank, because the app cannot invent it: the real branch list with coordinates and opening hours, counters and which service group each takes, the service catalogue with documents to bring, holiday calendar, priority categories, and the confirmed numbers in `domain/policy.ts` and `STAFF_POLICY`.

4. Data model

Tokens store facts, never the derived stage. That rule from the frontend carries over: `stage`, `ahead` and wait ranges are computed in a view or RPC from the facts, using the same rules as `domain/queue.ts` and `domain/staff.ts`.

Table	Holds	Key columns and constraints
`banks`, `branches`	Bank, branch, location	`branches`: `slug`, `lat`, `lng`, `timezone`, `address`, `hours` (jsonb), `holidays`; unique `(bank_id, slug)`
`service_groups`, `services`	Catalogue	`token_prefix` per group; `avg_service_min`, `bring` (text[]) per service
`counters`, `counter_groups`	Counters and which group each takes	unique `(branch_id, number)`
`profiles`	Customer profile and preferences	`id` = auth user id; `name`, `phone`, `email`, `alerts` (jsonb), `priority_tags` (text[]), `consent_at`

Table	Holds	Key columns and constraints
`tokens`	One row per visit	`code`, `number`, `branch_id`, `group_id`, `service_id`, `user_id` (null for walk-ins without an app), `source`, `priority` (text[]), `joined_at`, `delay_positions`, `lapse_count`, `checkin_prompted_at`, `checkin_confirmed_at`, `called_at`, `counter`, `on_my_way_at`, `extra_time_ms`, `extension_count`, `serving_started_at`, `served_at`, `missed_at`, `left_at`, `held`, `note`, `recalls`, `closed_by`, `feedback`, `booking_id`; unique `(branch_id, group_id, day, number)`
`token_sequences`	Next number per branch, group, day	Incremented inside the join RPC with a row lock so numbers never repeat
`token_events`	Append only history	`token_id`, `type` (joined, nearby, called, recalled, transferred, held, served, no_show, reopened, left), `actor`, `at`, `meta`
`bookings`, `slot_capacity`	Booked visits	unique `(branch_id, service_id, slot_start)` with a capacity check inside the booking RPC
`push_subscriptions`	Web Push endpoints	`user_id`, `endpoint` (unique), `keys`, `last_ok_at`; delete on 404 or 410
`staff`	Staff accounts	`employee_id` (unique), `branch_id`, `role`, `pin_hash` (argon2id), `failed_attempts`, `locked_until`, `disabled_at`
`rosters`	Who works which counter when	`staff_id`, `date`, `counter`, `shift_start`, `shift_end`
`shifts`, `breaks`	Open shift state and breaks	one open shift per counter per day; `break_reason`, `break_started_at`; `call_next_presses`, `nudge_snoozed_until`

Table	Holds	Key columns and constraints
`audit_log`	Staff and admin actions	`actor`, `action`, `entity`, `before`, `after`, `at`, `ip`

Views the apps read

- `token_view(token_id)`: stage, `ahead`, `now_serving`, estimated wait, counter, deadlines. Same fields as `TokenView` in `src/domain/types.ts`.
- `branch_snapshot` and `service_waiting`: public counts per service group, no personal data.
- `staff_queue(branch_id)`: waiting, called and serving entries in call order, with `confirming` for tokens whose check-in prompt is open.

Indexes that matter: a partial index on `tokens (branch_id, group_id, joined_at)` where `called_at is null and served_at is null and missed_at is null and left_at is null` (the live queue), `tokens (user_id, joined_at desc)` for history, and `token_events (token_id, at)`.

Row level security

Who	Can read	Can change
Anonymous	Branches, services, public counts	Nothing
Customer	Own profile, own tokens and bookings	Only through RPCs that check `auth.uid()`
Staff (JWT claims `staff_id`, `branch_id`)	Tokens and entries of their own branch, their own shift	Only through RPCs that check the claims and the open shift
Service role (Edge Functions)	Everything	Cron, push, admin tasks

Turn RLS on for every table, grant no direct insert, update or delete to `anon` or `authenticated`, and make every state change a `security definer` function with a pinned `search_path`.

5. Implementation phases and prompts

Build in nine phases, in this order, and finish each one's tests before starting the next. Each phase below has its goal, its steps and a prompt to paste into the other agentic IDE. Give the agent the repo and `docs/` every time; start every session with the context

prompt.

Context prompt (paste first, every session)

```
You are implementing the backend for QBIT9, a smart queue and appointment
system for Indian bank branches. The repo is a Next.js 16 app (customer app
at / and staff console at /staff) that runs today on a localStorage mock.

Read first: AGENT.md, docs/architecture.md, docs/backend-handoff.md,
docs/staff.md, src/services/types.ts, src/services/staff-types.ts,
src/domain/queue.ts, src/domain/staff.ts, src/domain/policy.ts.

Rules:
- Do not change screens, hooks or the service interfaces unless I say so. You
implement the interfaces in a new adapter at src/services/supabase/ and
register it in src/services/registry.ts.
- Business rules live in src/domain (pure, tested). Port them to SQL or Edge
Functions faithfully; do not invent new rules. Where a number is a policy
value, read it from a config table, never hardcode.
- Tokens store facts; the stage, ahead and wait are derived. Only staff
actions call a token. The customer's place is the real queue
(QueueContext.live in domain/queue.ts), never a timer.
- Every state change is a transactional Postgres function (security definer,
pinned search_path) that checks auth.uid() or the staff JWT claims and writes
a token_events row. No direct table writes from the client. RLS on for every
table.
- Throw the DomainError codes from src/domain/errors.ts so the UI copy keeps
working.
- No em dashes in copy or docs. No secrets in the repo. Commit migrations
under supabase/migrations.
- After each task, run: npm run check, and the tests for that phase. Report
what you changed and what is left.
```

Phase 0: Project setup

Goal: a Supabase project, local dev loop and CI, with the app still on the mock.

1. Create the Supabase project in the Mumbai region. Save the URL, publishable key, service role key and JWT secret in a password manager.
2. `npm i @supabase/supabase-js`, install the Supabase CLI, run `supabase init` and `supabase start` for a local stack.
3. Add `supabase/migrations`, `supabase/functions`, `supabase/seed.sql` and a GitHub Action that runs `supabase db lint`, applies migrations to a fresh local database and runs the tests.
4. Add the new env names from section 3 to `src/lib/env.ts` and `.env.example`.

```
Phase 0. Set up the Supabase project structure for QBIT9: supabase/ folder
with config, migrations, functions and seed.sql; a GitHub Actions workflow
that starts a local Supabase, runs migrations, runs pgTAP tests in
supabase/tests, then npm run check. Add the Supabase env vars to
src/lib/env.ts (read by full name only) and .env.example. Do not touch the
mock adapter. Show me how to run everything locally.
```

Phase 1: Schema, seed and RLS

Goal: every table from section 4 with constraints, indexes, RLS and seed data that matches the mock catalogue.

1. Write migrations for all tables, enums and the `token_sequences` locking function.
2. Port the catalogue from `src/services/mock/catalogue.ts` (banks, branches, groups, services, counters) into `seed.sql`. Use the same ids so existing tests and screenshots stay valid.
3. Write RLS policies exactly as in the table in section 4.
4. Create the views: `token_view`, `branch_snapshot`, `service_waiting`, `staff_queue`.

```
Phase 1. Create the Postgres schema for QBIT9 as migrations: banks, branches,
service_groups, services, counters, counter_groups, profiles, tokens,
token_sequences, token_events, bookings, slot_capacity, push_subscriptions,
staff, rosters, shifts, breaks, audit_log, plus a policy_config table holding
the values from src/domain/policy.ts and STAFF_POLICY per branch. Field names
follow TokenRecord in src/domain/types.ts and QueueEntry/EntryRecord in
src/domain/staff.ts. Add partial indexes for the live queue, enable RLS
everywhere with no direct write grants, and seed the catalogue from
src/services/mock/catalogue.ts keeping the same ids. Write pgTAP tests
proving a customer cannot read another customer's token and a staff JWT
cannot read another branch.
```

Phase 2: Branches and catalogue

Goal: `BranchService` (list, bySlug, byId, snapshot, snapshots, catalogue) on the real database.

1. Implement the adapter file `src/services/supabase/branches.ts` over the views.
2. Return live waiting counts per service from `service_waiting` (the real group queue, as the mock does when staff are in).
3. Opening hours and `isBranchOpen` come from branch hours and holidays, in the branch time zone.

```
Phase 2. Implement BranchService from src/services/types.ts in
src/services/supabase/branches.ts using supabase-js and the views from Phase
1. Match the return shapes the mock produces (see
src/services/mock/branches.ts and catalogue.ts). Waiting counts are the real
number of waiting tokens in the service's group queue. Open/closed uses the
branch's hours, holidays and timezone, never the browser clock. Add contract
tests that run the same assertions against the mock and the Supabase adapter.
```

Phase 3: Customer sign in and preferences

Goal: `AuthService` and `PreferencesService` on Supabase Auth.

1. Email OTP through Supabase Auth with a 4 digit code to match the UI (`OTP_LENGTH` in `domain/validation.ts`), or change that constant and the screen together. Phone OTP stays behind `NEXT_PUBLIC_FEATURE_SMS` .
2. `requestOtp` returns the challenge with no `debugCode` in real mode (the demo chip must not appear).
3. Rate limit OTP requests and attempts per identifier and per IP. Keep the mock's rules: resend gap, attempt cap, expiry.
4. `profiles` row is created on first verify. `deleteAccount` removes the profile, tokens' personal fields and push subscriptions, and keeps anonymised token rows for statistics.
5. `updateProfile` , `get` , `update` and `toggleSavedBranch` read and write `profiles` .

```
Phase 3. Implement AuthService and PreferencesService (src/services/types.ts)
in src/services/supabase/auth.ts and preferences.ts using Supabase Auth email
OTP. Behaviour must match src/services/mock/auth.ts: same DomainError codes
(otp_invalid, otp_expired, too_many_attempts, resend_too_soon, and so on),
same session shape, 30 day remember option, name collected at verify. No
debugCode in the real adapter. Add per-identifier and per-IP rate limits.
deleteAccount must erase personal data but keep anonymised tokens. Write
contract tests that run against both adapters, and a test that proves another
user's profile cannot be read.
```

Phase 4: Queue core (the hard part)

Goal: `QueueService` and the real queue. This phase decides whether the product works, so spend the most time here.

1. `join(input)` : inside one transaction, check the location proof (distance from branch, accuracy, age against `domain/geo.ts`), check the branch is open, reject if the user already has an active token (`already_in_queue`), take the next number with a row

lock on `token_sequences`, set `ahead` from the real waiting count of the group, place priority customers by the rules in `aheadForNewToken`, insert the token and a `joined` event.

2. `token_view`: port `deriveView` and `stageOf`. `ahead` is the count of waiting tokens in the same group that sort before this one, using `compareQueueOrder` (priority, then `order_at` = joined time plus setback, then number). `now_serving` is the most recently called token of the group.
3. Check-in: when `ahead <= nearThreshold` the prompt opens and lasts `checkinWindowMs`. A missed prompt sets the token back `setback` places (`delay_positions`) and reopens when near again. After `maxLapses` it becomes missed. Port this from `reconcileLive` in `domain/queue.ts`.
4. `confirmCheckin`, `moveBack`, `onMyWay`, `requestMoreTime` (max extensions), `leave`, `sendFeedback`, `active`, `get`.
5. The called window (`calledWindowMs` plus extensions) and no-show closure run on the cron function (Phase 7), not in the browser.
6. Staff cannot call a token while its check-in prompt is open. Enforce it in the call RPC.

```
Phase 4. Implement QueueService (src/services/types.ts) in
src/services/supabase/queue.ts backed by Postgres RPCs in
supabase/migrations. Port src/domain/queue.ts faithfully, in its LIVE mode
only (QueueContext.live): nobody is called by a timer, ahead comes from the
real queue ordered by compareQueueOrder in src/domain/staff.ts, check-in
prompt opens when ahead <= nearThreshold, a missed prompt sets the token back
by setback places, more than maxLapses means missed, the called window runs
in real time. The join RPC must be atomic: location proof check
(src/domain/geo.ts), branch open check, one active token per user, row-locked
number from token_sequences, priority placement from aheadForNewToken. Return
the same TokenView fields as deriveView. Write the SQL so token_view is a
view or function, not stored columns. Add pgTAP tests for every rule above
and a concurrency test: 50 simultaneous joins on one group must produce 50
unique numbers and a consistent order.
```

Phase 5: Bookings and history

Goal: `BookingService` and `HistoryService`.

1. `availability` from branch hours, holidays, the booking window and `slot_capacity` (see `domain/slots.ts`).
2. `create` takes a capacity lock inside one transaction and fails with `slot_full` when taken. Max active bookings per user as in the mock.

3. `checkInBooking` opens at the window defined by `bookingWindows` and creates a token with `source = booking`.
4. `cancel` frees the slot. `list` and `get` are scoped to the user.
5. `HistoryService.list` is a view over the user's closed tokens and bookings with the stats from `domain/history.ts`.

```
Phase 5. Implement BookingService and HistoryService in
src/services/supabase/. Port src/domain/slots.ts and bookingWindows exactly
(30 minute check-in window, booking horizon, closed dates, capacity). Slot
capacity must be enforced inside the create RPC with a lock so two people
cannot take the last slot. checkInBooking must create a token with source
booking that sorts by slot time. Contract tests against both adapters, plus a
concurrency test: 20 simultaneous bookings for a slot with capacity 3 produce
exactly 3 successes.
```

Phase 6: Staff sign in, shifts and counter actions

Goal: `StaffService` (`src/services/staff-types.ts`) on the real queue, so staff and customers share one queue.

1. Sign in goes through an Edge Function, never the client. It checks employee ID and PIN (argon2id hash in `staff_accounts`), locks the account for 5 minutes after 5 wrong tries, and returns a short lived JWT (8 hours) with claims `role=staff`, `staff_id`, `bank_id`, `branch_id`. RLS reads these claims.
2. Roster tables decide the counter and services at `startShift`. `startBreak`, `resumeFromBreak` and `endShift` update counter state. Counters on break are excluded from `waitingFor` estimates.
3. Every action is an RPC that checks the caller's branch and counter, then writes the token, a `token_events` row and an `audit_log` row:
   - `call_next`: closes the current token as served, picks the next by `compareQueueOrder`, refuses tokens whose check-in prompt is open.
   - `finish_current`, `recall` (max 2, only before arrival, fresh reply window), `transfer` (only to an open counter with the same service and nobody at it), `hold` (customer at counter), `resume` (clear counter), `set_note`, `snooze_nudge`.
   - `issue_walk_in` takes a number from the same `token_sequences` as customers.
   - `undo_closure` reverses an automatic no-show while the counter is clear.
4. Two staff pressing Call on the same token: exactly one wins, the other gets a clear `already_called` error.

```
Phase 6. Implement StaffService (src/services/staff-types.ts) in
src/services/supabase/staff.ts. Sign in is a Supabase Edge Function: argon2id
PIN check, lockout after 5 wrong tries for 5 minutes, returns an 8 hour JWT
with role, staff_id, bank_id, branch_id claims. Write RPCs for every counter
action in the StaffService contract, each checking the caller's branch from
the JWT, each writing token_events and audit_log. Port the transition rules
from src/domain/staff.ts (recall limit 2, hold and resume conditions,
transfer conditions, order by compareQueueOrder, nudge after max(5 min, 2 x
usual service time)). Staff walk-ins and customers use the same
token_sequences row. Add pgTAP tests: a staff member from branch A cannot
read or act on branch B, double call on one token yields one success,
call_next skips nothing and respects priority, lockout works, an expired JWT
is rejected.
```

Phase 7: Realtime, push and timers

Goal: the app updates live and customers are reached when the tab is closed.

1. `subscribe` in `QueueService` and `StaffService` uses Supabase Realtime on `tokens` (filtered by user or by branch) and refetches the view. Never trust the payload alone, refetch through `token_view`.
2. Web Push with VAPID keys. Store `push_subscriptions` per user and device. Send on: called, recall, check-in prompt opened, check-in missed, no-show closure. iOS needs the app added to the Home Screen, so show that guidance in the UI.
3. A `pg_cron` job runs every 15 to 30 seconds: expire the called window, open or lapse check-in prompts, close no-shows, release expired location proofs. It writes events, and a trigger or Edge Function sends the push.
4. Email through Resend for booking confirmations and reminders. SMS stays behind `NEXT_PUBLIC_FEATURE_SMS`.

```
Phase 7. Implement subscribe() for QueueService and StaffService with
Supabase Realtime (refetch on change, debounce 200 ms). Implement Web Push:
service worker, VAPID keys from env, push_subscriptions table, an Edge
Function send-push called from database triggers on token_events (called,
recall, checkin_open, checkin_missed, no_show). Implement a pg_cron job every
30 seconds that advances time based rules: called window expiry, check-in
window expiry and setback, no-show closure. The job must be idempotent and
safe to run twice at once (use advisory lock). Add tests: a token called with
no tab open triggers one push, the cron job run twice produces one event,
Realtime delivers a staff call to the customer within 2 seconds on staging.
```

Phase 8: Wire the adapter into the app

Goal: switch the app from mock to Supabase with no screen changes.

1. Implement `createSupabaseServices()` in `src/services/supabase/index.ts` returning the full `Services` object including `staff`.
2. In `src/services/registry.ts` replace the throwing `supabase` branch with that factory.
3. Set `NEXT_PUBLIC_QBIT9_ADAPTER=supabase` in staging first. `staffDev` must be undefined, so the demo tools and DevPanel disappear. Confirm no code outside `services/mock` imports the mock (ESLint already enforces it).
4. Remove the mock from the production bundle: guard the dynamic import so the bundle analyser shows no `services/mock`.
5. Run the contract test suite against both adapters. Any difference is a bug in the new adapter, not in the screens.

```
Phase 8. Implement createSupabaseServices() in src/services/supabase/index.ts and enable it in src/services/registry.ts for NEXT_PUBLIC_QBIT9_ADAPTER=supabase. Do not change any file in src/features, src/hooks or src/app unless a contract gap is proven, and if so, change the contract in src/services/types.ts and both adapters together. staffDev stays undefined. Verify with the production build that nothing from src/services/mock is bundled, and that the demo chip, demo tools and DevPanel are absent. Run unit tests, contract tests and the Playwright suite against staging.
```

Phase 9: Hardening

Goal: make it safe to put in front of a real branch.

1. Run the security checklist (section 8) and the production checklist (section 9).
2. Load test one busy branch: 300 customers joining over 10 minutes with 6 counters calling.
3. Chaos checks: kill the cron job, drop the network on a customer phone, restart the database.
4. Pilot with one branch behind a feature flag and a manual fallback (paper tokens) before widening.

```
Phase 9. Run a hardening pass. Produce a report listing each item in the
security checklist and production checklist with status and evidence. Fix
everything marked high. Add a k6 load script for one branch (300 joins over
10 minutes, 6 staff counters calling), record p95 latency for join, get and
call_next, and fail the build if p95 exceeds 500 ms. Add alerts for error
rate, cron job lag and push failure rate.
```

6. Integrating the adapter into the current app

The app was built so that this step is small. Screens only know hooks, hooks only know the contracts in `src/services/types.ts` and `src/services/staff-types.ts`. The backend plugs in at one place.

Step	Where	What
1	`src/services/supabase/`	One file per service, plus `index.ts` exporting `createSupabaseServices(): Services`.
2	`src/services/registry.ts`	Replace the throwing `supabase` branch with the factory. Selected by `NEXT_PUBLIC_QBIT9_ADAPTER`.
3	`src/lib/env.ts`	Flags: `backgroundAlerts` true, `devtools` false, `staffApp` per environment.
4	`src/hooks/`	Unchanged. `subscribe` now comes from Realtime instead of the localStorage change event.
5	`src/domain/`	Unchanged and reused by the adapter. Rules are ported to SQL, and the TypeScript domain stays as the reference that contract tests compare against.
6	Mock	Stays in the repo for tests and demos, excluded from the production bundle.

Rules for the integration

- If a screen needs a change, the contract was wrong. Fix the contract and both adapters together, never one screen.
- Errors cross the boundary as `DomainError` with a stable `code`. Map every Postgres and RPC error to one, so screens never see raw database text.
- Times cross the boundary as ISO strings in UTC. The app formats in `Asia/Kolkata`.
- The server is the clock. Never trust the browser's time for queue rules, only for countdown display.

- Rollout is by flag: staging on `supabase`, production on `mock` until the pilot branch is ready, then `supabase` with the staff app on for the pilot only.
- Keep the DevPanel and `staffDev` out of production. Verify by searching the built bundle for `services/mock`.

Common traps

- Realtime delivers a change, not a truth. Always refetch through `token_view` rather than patching the cache from the payload.
- Two tabs for one customer must stay consistent. Mutations invalidate, they do not write local state.
- Location proofs expire. Fail the join with a clear message, not a silent retry.
- Do not store derived stage in a column. The view derives it, as in the mock.

7. Test cases

Existing coverage to keep green: 207 unit tests (domain, mock services) and 12 Playwright tests. The tests below are what the backend must add. The key idea: **one contract suite runs the same assertions against the mock and the Supabase adapter.** If both pass, the screens cannot tell them apart.

7.1 Contract tests (mock vs Supabase)

Area	Cases
Auth	request OTP, wrong code, expired code, attempt cap, resend too soon, sign out, delete account erases personal data
Branches	list, by slug, snapshot numbers match the waiting count, catalogue excludes services with no open counter
Queue join	valid proof joins; rejects far away, poor accuracy, stale proof, closed branch, second active token
Queue live	ahead matches real waiting count, priority goes first, setback moves a token back by `setback` places, lapse after the window, missed after `maxLapses`
Queue actions	confirm check-in, on my way, more time up to the limit, leave, feedback after close

Area	Cases
Bookings	availability respects closed dates and horizon, capacity enforced, cancel frees the slot, check-in only inside the window
History	closed tokens and bookings listed with correct stats
Staff	sign in, lockout after 5 tries, call next, finish, recall limit 2, hold and resume, transfer rules, walk-in, undo closure, break changes waits

7.2 Database tests (pgTAP)

- RLS: a customer cannot read or write another customer's token, booking, profile or push subscription.
- RLS: staff from branch A cannot read or act on branch B.
- RLS: the anonymous role reads only public branch data.
- Constraints: one active token per user, unique token number per group per day, booking capacity.
- Functions: every RPC fails cleanly for the wrong role.

7.3 Concurrency

- 50 simultaneous joins on one group give 50 unique, gap free numbers.
- Two staff call the same token: one success, one `already_called`.
- Two staff each call next on the same group: two different tokens, no skip, no duplicate.
- 20 bookings for a slot with capacity 3: exactly 3 succeed.
- Cron job run twice at once: one event per transition.

7.4 Timers and delivery

- Called window expires, no-show closes, and the customer gets one push.
- Check-in prompt opens at the threshold, lapses after the window, reopens after setback.
- Push arrives with the tab closed on Android Chrome, and on iOS with the app on the Home Screen.
- A staff Call reaches the customer through Realtime within 2 seconds.

7.5 End to end (Playwright, staging)

Run each at Pixel 7 and 1280x800.

1. Customer: sign in, find a branch, join, wait, check in, get called, finish, give feedback.
2. Staff: sign in, start shift, call next, serve, recall, transfer, break, end shift.
3. Cross: customer joins in one browser, staff calls in another, customer sees "Your turn".
4. Booking: book, see it in history, check in at the window, cancel another.
5. Failure: offline during join, expired session, OTP errors, branch closed.

7.6 Load and resilience

- 300 joins over 10 minutes on one branch with 6 counters, p95 under 500 ms for join, get and call next.
- 50 branches live at once with a Realtime connection per customer.
- Restart the database during a run and confirm clients recover with no duplicate tokens.

7.7 Security and quality

- Attempt RLS bypass with the anon key and with another user's JWT.
- Confirm the service role key never appears in the client bundle or network calls.
- Injection and XSS on name, note and feedback fields.
- Accessibility: keyboard flow, screen reader labels, contrast, 44 px touch targets, reduced motion.
- Slow 3G and offline: join fails with a clear message and never double submits.

8. Security and privacy checklist

Tick each item with evidence (a test, a screenshot, a config link) before the pilot.

Access control

- [ ] RLS enabled on every table, default deny, no table without a policy.
- [ ] Customers see only their own rows. Staff see only their branch. Anonymous sees only public branch data.
- [ ] All writes go through RPCs or Edge Functions, not direct table writes from the client.
- [ ] Service role key exists only in server secrets and CI, never in `NEXT_PUBLIC_*`, never in the bundle.
- [ ] Staff JWT is short lived (8 hours), carries role and branch claims, and is checked in every RPC.

Authentication

- [ ] OTP rate limits per identifier and per IP, with attempt cap and expiry.
- [ ] Staff PIN stored as argon2id, 5 wrong tries lock for 5 minutes, lockouts are logged.
- [ ] Sessions can be revoked. Sign out clears push subscriptions for that device.

Data protection (India, DPDP Act)

- [ ] Collect only what is needed: name, contact, location proof for the join check. Location proof is not stored after validation, only the pass or fail and the distance bucket.
- [ ] Clear consent text at sign in, with a link to the privacy policy.
- [ ] Delete my account works end to end and is tested. Anonymised tokens remain for statistics.
- [ ] Retention rule agreed with the bank (for example personal fields on closed tokens removed after 90 days).
- [ ] Data stays in the Mumbai region. Backups too.
- [ ] Name a grievance contact and a breach response owner.
- [ ] Audit log for every staff action, append only, readable by bank admins only.

Application

- [ ] CSP, HSTS, X-Content-Type-Options and Referrer-Policy headers set. No inline script without a nonce.
- [ ] All user text is escaped on render. Validate with zod on the client and again in the database.
- [ ] CORS limited to the app origins.
- [ ] Dependency and secret scanning in CI (npm audit, Dependabot, gitleaks).
- [ ] Error messages shown to users never include database text.

Operations

- [ ] Secrets in a manager, rotated on a schedule, different per environment.
- [ ] Point in time recovery on, and one restore drill done.
- [ ] Least privilege for people: separate accounts, 2FA on every vendor console.

9. Production readiness checklist

Environments and delivery

- [ ] Three environments: local, staging, production, each with its own Supabase project, keys and Vercel scope.
- [ ] CI on every PR: typecheck, lint, unit tests, contract tests, pgTAP, build, Playwright against a preview.

- [ ] Migrations are files in the repo, applied by CI, never edited in the dashboard. Every migration has a rollback note.
- [ ] Production deploys are manual approval, with instant rollback to the previous build.

Reliability

- [ ] Sentry for browser, Edge Functions and the cron job, with release tracking.
- [ ] Uptime check on the app and on a `/health` endpoint that touches the database.
- [ ] Alerts: error rate, p95 latency, cron job lag over 60 seconds, push failure rate, database connections.
- [ ] Structured logs with token and branch ids, no personal data.
- [ ] Load test passed (section 7.6) and results saved.

Product

- [ ] Real data loaded: banks, branches, hours, holidays, services, counters, staff roster, geofence radius.
- [ ] Demo chip, demo tools, DevPanel and sample staff accounts are gone from production.
- [ ] Policy numbers in `domain/policy.ts` and `STAFF_POLICY` replaced by values the bank confirmed.
- [ ] Installable PWA, push permission flow, and iOS Home Screen guidance tested on real phones.
- [ ] Fallback when location is denied or push is blocked, with plain copy.
- [ ] Manual fallback if the system is down: paper tokens and a staff instruction card.

Messaging and domain

- [ ] Domain with HTTPS, SPF, DKIM and DMARC set for the email sender.
- [ ] SMS needs DLT registration (entity, sender id, templates). Start early, it takes weeks.
- [ ] Privacy policy and terms published and linked in the app.

Rollout

1. Staging with the bank's real branch data and internal testers.
2. One pilot branch, staff trained, a bank contact on call, daily review for two weeks.
3. Widen to a few branches, then a region.
4. Written runbooks: queue stuck, push not arriving, staff cannot sign in, database failover, rollback.

10. Open decisions to settle with the bank

Decision	Why it matters	Current assumption
Policy numbers (near threshold, check-in window, setback, called window, max extensions, max lapses)	Every customer promise depends on them	Sample values in `domain/policy.ts`
Recall and nudge values	Staff workload and fairness	Recall max 2, nudge after max(5 min, 2 x usual time)
Geofence radius and accuracy limit	Stops remote joining without blocking real customers	Values in `domain/geo.ts`
Where staff rosters come from	Counter and service assignment at shift start	Entered by a bank admin in a simple table
Staff sign in method	PIN is simple, passkeys are stronger	PIN now, passkey later
QR or sticker scan at the counter	Proves arrival without trusting the phone	Not built
Printed slip for walk-ins	Customers without phones	Not built, staff walk-in screen only
SMS vs push vs email	Cost and reach, especially on iOS	Push plus email, SMS behind a flag
Data residency and retention	DPDP compliance, bank audit	Mumbai region, 90 day personal field retention
Service level	Uptime and support hours expected by the bank	99.5 percent in branch hours, to be agreed
Pricing and ownership of the Supabase, Vercel and messaging accounts	Who pays and who holds the keys	Navchetna owns them for the pilot